

STREAMFLIX - Movie & Series Streaming Platform Database

Database Management Systems (COMP2004) - Term Project

Team: Goktug Karaca, Defne Kaya Date: 29 May 2026

1. System and Requirements

STREAMFLIX is the database for a movie and series streaming service like Netflix. Users sign up with their own details, log in, and watch films and series. They can add titles to a watchlist, mark favorites, keep a watch history, and rate or review what they watch. We only built the database layer; there is no web or mobile app. The main requirements are:

- Users register with personal info (email, name, country, birth date) and log in.
- An account can hold several profiles, including a kids profile.
- Users pick one of three plans (Basic, Standard, Premium) and pay monthly.
- Content is either a movie or a series; series have seasons and episodes.
- Each title has one or more genres and a cast of actors and directors.
- Profiles keep a watchlist, favorites, watch history, and reviews with a rating.
- We also store subtitle languages per title and the devices each user signs in from.

2. Logical Design (E-R Model)

The system has 22 tables. All tables, keys, and relationships are defined in schema.dbml and drawn as an E-R diagram on dbdiagram.io. The main relationships are:

Relationship	Cardinality	Note
users - profiles	1 : N	One account, many profiles
users - subscriptions - payments	1 : N : N	Subscription and payment chain
content - movies / series	1 : 1	Supertype / subtype split
series - seasons - episodes	1 : N : N	Series hierarchy
content - genres	N : M	via content_genres
content - people	N : M	via content_cast (with role)
profiles - content	N : M	via watchlist, favorites, reviews

2.1. Normalization (1NF to 3NF)

To show the design avoids redundancy, here is how a single flat table would be broken down. Start with one wide table:

Watch(user_email, user_name, country_name, country_code, plan_name, plan_price, title, genres, director, rating)

- **Anomalies:** Insertion - cannot add a plan or a title until someone uses it. Update - changing a plan price means editing every matching row. Deletion - removing the last view of a title loses the title itself.
- **Functional dependencies:** user_email -> user_name, country_name; country_name -> country_code; plan_name -> plan_price; content_id -> title, director.
- **1NF:** genres is multi-valued, so it moves to its own rows: content_genres(content_id, genre_id).
- **2NF:** Partial dependencies are removed by splitting out users, subscription_plans and content.
- **3NF:** Transitive dependencies are removed: country_name -> country_code goes to countries; plan_price goes to subscription_plans; director goes to people + content_cast.

After this, all 22 tables are in 3NF: every non-key column depends fully and directly on its primary key. Most tables are also in BCNF. The decomposition is lossless and dependency preserving.

3. Physical Design

01_create_tables.sql builds every table on MySQL 8.0. Constraints used:

- **Primary keys:** on every table; composite keys on link tables (e.g. content_genres).
- **Foreign keys:** on all relationships, with ON DELETE CASCADE where it makes sense (deleting a user also removes their profiles, subscriptions and devices).
- **UNIQUE:** users.email, genres.genre_name, and reviews(profile_id, content_id) so a profile reviews a title once.

- **NOT NULL / CHECK:** rating BETWEEN 1 AND 10, progress_percent BETWEEN 0 AND 100, content_type IN ('Movie','Series'), payment_status IN ('Success','Failed','Refunded'), and similar.

Sample data: 02_sample_data.sql loads 426 rows across the 22 tables (15 users, 26 profiles, 20 titles, 32 episodes, 28 history rows, 20 reviews, and so on). All rows load cleanly with foreign key checks on.

4. SQL Implementation

Every statement has a short business question above it.

- **03_dml_operations.sql:** 14 statements - 5 INSERT, 5 UPDATE, 4 DELETE (new signup, subscription, profile, plan downgrade, expiring subscriptions, account deletion with cascade, and more).
- **04_queries.sql:** 5 simple queries (filtering and sorting) and 7 complex ones using JOIN, GROUP BY / HAVING, and subqueries: average rating per title, busiest genres, total paid per user, titles above the average rating, titles never watched, top 5 favorites, and titles by a given director.

Testing and demo: All files (schema, data, DML and the 12 queries) were run on a real MySQL 8.4 server with no errors. The live demo runs in MySQL Workbench in order 01 to 04; the E-R diagram is shown on dbdiagram.io from schema.dbml.